CSRF Security Testing Methods

Educational Bug Bounty Series • Authorized Security Research Only

■ Cross-Site Request Forgery (CSRF) — Testing Methods

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Use a lab or a dedicated test account. Do not trigger state-changing requests for other users or publish live CSRF pages targeting real services.*

■ Executive Summary

CSRF occurs when a browser sends an authenticated request that causes an unwanted state change because the application does not adequately verify that the request was intentionally initiated by the user. The supplied three-page *CSRF 1.pdf* lists missing or weak tokens, referer/content-type checks, request-method variations, token reuse, and token/session-binding concerns.

■ Core Testing Methodology

1. **Identify state-changing actions.** Review email changes, profile updates, password changes, and other operations that use ambient browser credentials.
2. **Inspect defenses.** Check CSRF tokens, `Origin`/`Referer` validation, content-type rules, and cookie `SameSite` settings.
3. **Check token enforcement.** In a controlled test, verify that missing, invalid, or expired tokens are rejected.
4. **Check session binding.** Confirm a token issued to one test session cannot authorize a different session.
5. **Check request-method consistency.** Ensure security checks apply to every supported method and that state-changing actions are not exposed through unsafe GET routes.
6. **Check token lifecycle.** Verify token rotation and invalidation behavior matches the application's design.
7. **Review referer/origin policy.** Confirm validation is robust when headers are absent or malformed; do not rely on a single weak signal.
8. **Review cookie behavior.** Check whether cookies are sent cross-site and whether `SameSite` policy is appropriate.
9. **Record outcomes.** Use a reversible test action and redacted evidence.
10. **Retest remediation.** Add regression tests for all affected actions.

■ High-Value Bug Bounty Tips & Edge Cases

- **Missing or weak tokens:** A token must be unpredictable, validated server-side, and bound to the user session.
- **Content type:** Do not assume a content-type check alone is sufficient protection.
- **Referer/Origin checks:** Validate the expected origin carefully and define behavior for missing headers.
- **Method changes:** State-changing actions should not become available through GET simply because another method is protected.
- **Token presence-only validation:** Presence is not enough; the token must be valid and correctly bound.
- **Token reuse:** Reuse policy should be explicit. A token should not be accepted across unrelated sessions.
- **Method override:** Ensure method-override features cannot bypass CSRF protections.
- **Cookie/header token patterns:** If using a double-submit pattern, bind and validate the token correctly; do not merely check that two arbitrary values exist.
- **Source limitation:** The PDF's examples are proof-of-concept sketches and do not document a specific target's behavior.

■■ Educational Checklists & Safe Examples

```
Action: [Controlled-Test-Action]
Session: [Test-Session-A]
Token state: [Missing / Invalid / Expired / Valid]
Expected: [Invalid requests rejected]
Observed: [Redacted result]
State changed: [Yes/No]
```

- [ ] All state-changing endpoints require CSRF protection where applicable.
- [ ] Tokens are unpredictable and session-bound.
- [ ] Origin validation is correctly configured.
- [ ] GET requests do not perform sensitive state changes.
- [ ] Method overrides cannot bypass protections.
- [ ] Cookies use appropriate `SameSite` settings.
- [ ] Tests use reversible actions and controlled accounts.

■■ Mitigation & Remediation

- Use framework-provided CSRF middleware.
- Validate synchronizer tokens or correctly implemented double-submit tokens.
- Validate `Origin` and, where appropriate, `Referer` as defense in depth.
- Avoid state changes through GET.
- Set cookies with suitable `SameSite`, `Secure`, and `HttpOnly` attributes.
- Ensure all supported content types and method-override paths receive equivalent protection.
- Add regression tests for missing, invalid, expired, and cross-session tokens.

■ Source Notes

Based on the supplied three-page *CSRF 1.pdf*. Page 1 lists general bypass categories and a basic form example; page 2 discusses token validation tied to presence or request method, session binding, method override, and duplicated token/cookie patterns. Page 3 has no additional readable text in the supplied preview. The guide summarizes concepts without reproducing a ready-to-use cross-site attack page.

Educational use only. Stay within scope and disclose findings responsibly.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.